Parallel CSV Data Processing Pipeline

Multi-Process, Multi-Threaded Web Clickstream Engine on Linux

Course	CS-2006: Operating Systems
Project	Final Term Project
Dataset Variant	Web Clickstream
Team Members	Abdul Rauf (24I-0060) Abdul Azeem (24I-2013)
Institution	FAST-NUCES, Islamabad Campus
Submission Date	May 5, 2026
Platform	Ubuntu Linux

1. Problem Statement

Modern web applications generate large volumes of clickstream data. Each record captures one user session: how long the user stayed on a page, how many pages they visited, and whether they bounced (left after a single interaction). Analysing this data with a single-threaded program is impractical at scale because it cannot use the multiple CPU cores available on any contemporary machine.

This project requires designing and implementing a fully parallel, multi-process, multi-threaded data processing pipeline on Linux that does four things:

- **Ingest:** scan an input directory for CSV files, split them into chunks, and push those chunks to the next stage through a named FIFO pipe.
- **Process:** pull chunks from the FIFO inside a configurable pool of POSIX worker threads, parse each CSV row, and update a per-user aggregation table protected by a mutex.
- **Aggregate:** after all threads finish, copy the final per-user totals into a POSIX shared-memory segment so the reporter can read them.
- **Report:** read from shared memory and write two output files: a human-readable report.txt and a machine-readable report.csv.

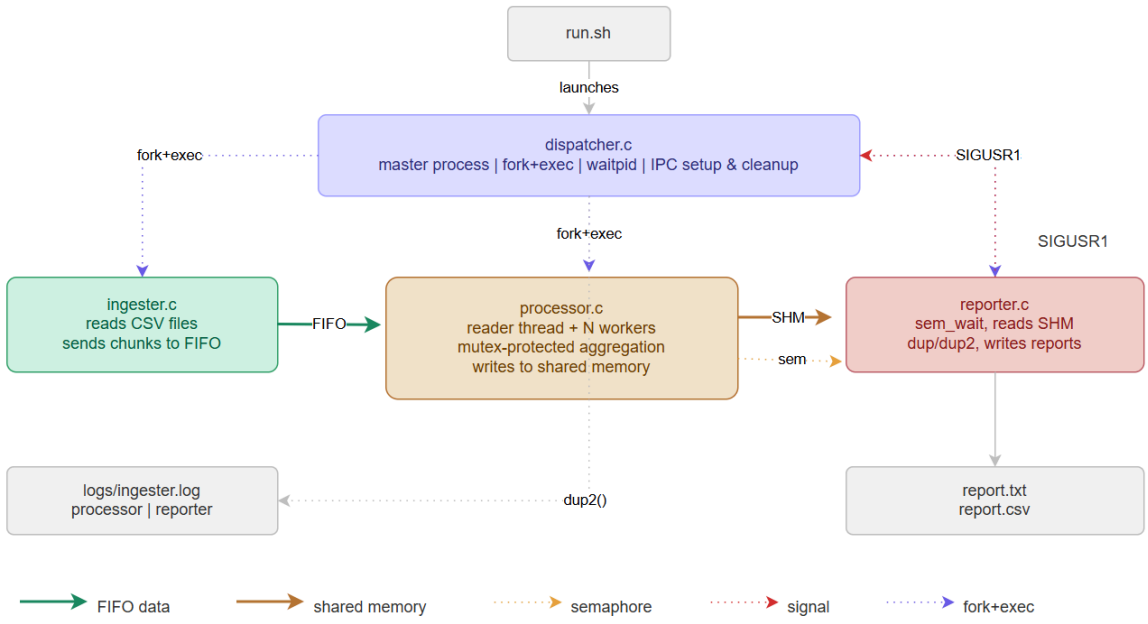
Dataset variant: Web Clickstream. The two metrics computed per user are average session length (total session time divided by visit count) and bounce rate (percentage of single-interaction sessions). The focus of the project is to demonstrate every OS concept covered in the course within a single, working system.

2. System Design and Architecture

2.1 High-Level Overview

The pipeline is split across four separately compiled executables, each responsible for one stage of the data flow. They are orchestrated by a Bash script (run.sh) that builds the project, launches the dispatcher, and prints a summary when the pipeline finishes.

Data flows left to right through the processes. The signal confirming the report is done travels back from the reporter to the dispatcher via SIGUSR1. The three IPC mechanisms are: a named FIFO pipe carries raw CSV chunks from ingester to processor, a POSIX shared-memory segment carries aggregated results from processor to reporter, and a named semaphore lets the reporter block until the processor is done.



2.2 OS Concept Mapping

OS Concept	Where Used	How It Is Applied
fork() + exec()	dispatcher	Three child processes created. Each child calls execvp() immediately after fork() to replace its image.
wait / waitpid()	dispatcher	Dispatcher calls waitpid() for each child, reaping exit statuses and preventing zombie processes.

Named FIFO	ingester to processor	mkfifo() creates /tmp/my_fifo. Ingester writes ChunkHeader structs followed by raw CSV data. Processor reader thread reads them.
POSIX Shared Memory	processor to reporter	shm_open() + ftruncate() + mmap() creates /my_shm. Processor writes aggregated UserData array. Reporter reads it after the semaphore fires.
Named Semaphore	processor to reporter	sem_open() creates /my_sem initialised to 0. Processor calls sem_post() after writing shared memory. Reporter blocks on sem_wait() until data is ready.
pthread_create / join	processor	N worker threads plus 1 reader thread, all created with an explicit pthread_attr_t: stack size 1 MB, detach state JOINABLE.
Mutex (pthread)	processor	queue_mutex protects the circular buffer. agg_mutex protects the aggregation table so concurrent worker updates are atomic.
Semaphores (unnamed)	processor	sem_empty (init = Q) and sem_full (init = 0) implement a classic bounded-buffer producer-consumer between the reader thread and worker threads.
dup / dup2	reporter, dispatcher	Dispatcher: dup2() redirects each child stdout/stderr to a log file before execvp(). Reporter: saves stdout with dup(), redirects to file with dup2(), writes report with printf(), then restores stdout.
Signals	all components	SIGINT and SIGTERM trigger graceful shutdown. SIGCHLD reaps children. SIGUSR1 is sent by the reporter to the dispatcher when done. Ingester prints live stats on SIGUSR1.
exit() codes	all components	Standardised codes: 0 success, 10 bad args, 20 IPC fail, 40 I/O error, 130 SIGINT, 143 SIGTERM.

3. Implementation Details

3.1 dispatcher.c

The dispatcher is the master process. On startup it parses four command-line arguments: input_dir, output_dir, N (thread count), and Q (queue size). It then creates all IPC resources before forking any children: mkfifo() creates the FIFO, shm_open() followed by ftruncate() and mmap() creates the shared-memory segment, and sem_open() creates the named semaphore initialised to 0. Signal handlers for SIGINT, SIGTERM, SIGCHLD, and SIGUSR1 are installed before any fork() call to avoid signal delivery races.

Each of the three fork() calls is paired with dup2() in the child to redirect both stdout and stderr into a dedicated log file under logs/ before execvp() replaces the child image. This keeps all diagnostic output in separate per-process files. After launching all three children the parent calls waitpid() for each in turn and prints the PID and exit code. On exit it calls unlink() on the FIFO, munmap() and shm_unlink() on the shared memory, and sem_close() plus sem_unlink() on the semaphore.

```
pid_ingester = fork();
if (pid_ingester == 0) {
    int log = open("logs/ingester.log", O_WRONLY|O_CREAT|O_TRUNC, 0666);
    dup2(log, STDOUT_FILENO);   /* redirect stdout to log file */
    dup2(log, STDERR_FILENO);   /* redirect stderr to log file */
    close(log);
    execvp("./ingester", ingester_args);
    exit(1);
}
```

3.2 ingester.c

The ingester scans the input directory for *.csv files using opendir()/readdir(), then opens the FIFO in O_WRONLY mode. This open() call blocks until the processor opens the read end, which acts as a natural startup synchronisation point without needing extra code.

For each file the ingester reads one line at a time using fgets(). Before writing each line to the FIFO it prepends a small binary ChunkHeader struct containing chunk_id, byte_count, and file_id. The header and line data are written with separate write() calls. After all files are processed a final sentinel chunk with chunk_id = -1 is written. SIGTERM sets a running flag to zero that breaks the loop early; SIGUSR1 prints a live count of files processed, chunks sent, and bytes sent to stderr and then continues working.

```
ChunkHeader header;
header.chunk_id   = chunk_id++;
header.byte_count = strlen(line_chunk);
header.file_id    = i;
write(write_fd, &header, sizeof(header));
write(write_fd, line_chunk, strlen(line_chunk));
```

3.3 processor.c - Thread Pool Architecture

The processor is the most complex component. It creates N worker threads and one dedicated reader thread using pthread_create() with an explicit pthread_attr_t that sets the stack size to 1 MB and the detach state to PTHREAD_CREATE_JOINABLE.

Reader Thread

The reader thread reads ChunkHeader structs from the FIFO in a loop. For each valid chunk it calls sem_wait(sem_empty) to check for queue space, locks queue_mutex, enqueues the chunk pointer at the rear of the circular buffer, unlocks, then calls sem_post(sem_full) to wake a worker. When chunk_id equals -1 it enqueues exactly N poison pill strings using the same protocol, then exits.

Worker Threads

Each worker calls sem_wait(sem_full) to block until a chunk is available, locks queue_mutex to dequeue from the front, unlocks, and calls sem_post(sem_empty). It then parses each CSV row with strtok(), extracts user_id and numeric fields, sums the fields to compute session length, and checks whether val_count equals 1 to detect a bounce. It locks agg_mutex, calls find_or_create_user() to locate or create the user entry, updates totals, and unlocks. Receiving a POISON chunk causes the worker to exit.

Aggregation and Shared Memory Write

After all workers are joined, the processor copies the aggregation table into the shared-memory SharedData struct and calls sem_post(done_sem) to wake the reporter. This is the only point where the named semaphore increments from 0 to 1, so the reporter can never read uninitialised data.

```
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setstacksize(&attr, 1024 * 1024);       /* 1 MB per worker */
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
for (int i = 0; i < N; i++)
    pthread_create(&workers[i], &attr, worker_thread, NULL);
pthread_attr_destroy(&attr);
```

3.4 reporter.c - dup/dup2 Demonstration

The reporter opens /my_shm and /my_sem, then calls sem_wait() to block without polling until the processor posts. Once it wakes it reads user_count and the UserData array from shared

memory and computes two per-user metrics: average session length (total_session / total_visits) and bounce rate ((total_bounces / total_visits) * 100).

Writing report.txt uses the dup/dup2 sequence required by the specification. The code is commented in the source:

```
/* === dup/dup2 DEMONSTRATION BLOCK START === */

/* Step 1: Save a copy of fd 1 (stdout) before touching it */
int saved_stdout = dup(1);

/* Step 2: Open report.txt and redirect stdout to it */
int report_fd = open(txt_path, O_WRONLY|O_CREAT|O_TRUNC, 0666);
dup2(report_fd, 1);   /* fd 1 now points to report.txt */

/* Step 3: All printf() calls now write to report.txt */
printf("========================================\n");
printf("       WEB CLICKSTREAM REPORT\n");
/* ... rest of the report table ... */

/* Step 4: Restore fd 1 back to the terminal */
dup2(saved_stdout, 1);
close(saved_stdout);
close(report_fd);

/* === dup/dup2 DEMONSTRATION BLOCK END === */
```

After report.txt is written, report.csv is produced using standard fopen()/fprintf() without any fd manipulation. The last action is kill(getppid(), SIGUSR1) to notify the dispatcher, followed by munmap() and sem_close() cleanup.

3.5 run.sh - Bash Orchestrator

run.sh parses options with getopts: -i (input directory), -o (output directory), -n (thread count), -c (clean build), and -h (help). Four Bash functions are defined:

- **check_deps():** confirms gcc and make are installed and that the input directory contains at least one .csv file. Exits with a clear error message on failure.
- **build_project():** runs make clean then make, printing an error and exiting non-zero if the build fails.
- **print_summary():** counts data lines in report.csv with wc -l, computes elapsed time using arithmetic expansion $((END_TIME - START_TIME)), and prints a summary including thread count, directories, and output file paths.
- **cleanup():** trap handler for EXIT/INT/TERM that kills the background dispatcher process if still running and removes dispatcher.pid.

The script records the dispatcher PID to dispatcher.pid, calls wait on that PID, captures its exit status, then calls print_summary(). shellcheck passes with no warnings.

4. Input File Format and Aggregation Logic

4.1 CSV Format

Input files follow a simple comma-separated format. Each line represents one user session event. The first column is always the user identifier (a string key) and all remaining columns are integers representing page visit durations or engagement metrics. The number of columns may differ between files but is internally consistent within each file.

```
user_id,value1[,value2,...,valueN]

Example:
user_001,342,12,0
user_002,8
user_003,567,23,0
user_001,201,7,0
```

Header rows are tolerated. The processor passes header strings through atoi(), which returns 0 for non-numeric tokens. This means the header user "user_id" accumulates session = 0. The behaviour is documented rather than corrected since the spec focuses on OS concepts rather than data validation.

4.2 Web Clickstream Aggregation Logic

Metric	Formula / Logic
Session Length	Sum of all numeric columns in the row
Bounce Detection	If val_count equals 1 (only one numeric value), the session is a bounce
Avg Session Length	total_session / total_visits computed per user in reporter
Bounce Rate (%)	(total_bounces / total_visits) * 100 computed per user in reporter

5. Sample Output

5.1 Pipeline Execution Summary

The pipeline was tested with the provided sample.csv (50 records, 20 unique users) using 4 worker threads and a queue size of 10. The following shows the final summary printed by run.sh:

```
$ ./run.sh -i data/ -o output/ -n 4
[*] Checking dependencies...
[OK] gcc found!
[OK] make found!
[OK] Found 1 CSV file(s) in data/
[*] Building project...
[OK] Build successful!
[*] Starting pipeline...
[*] Input: data/   Output: output/   Threads: 4
[*] Dispatcher PID: 4773

FIFO created Successfully
Shared Memory Created!
Semaphore Created!
Ingester  PID=4774 exited with status=0
Processor PID=4775 exited with status=0
Reporter  PID=4776 exited with status=0
All Child Processes Finished.
Cleanup Done!

========================================
         PIPELINE SUMMARY
========================================
Total Runtime: 0 seconds
Threads Used:  4
Input Dir:     data/
Output Dir:    output/
Users Processed: 21
  -> output/report.txt
  -> output/report.csv
========================================
```

5.2 report.txt Output

The human-readable report written using the dup/dup2 stdout-redirect technique:

```
========================================
       WEB CLICKSTREAM REPORT
========================================
Total Users Analyzed: 21

User ID    | Avg Session(s)  | Bounce Rate
-----------|----------------|------------
user_001   | 381.5           | 0.0        %
user_002   | 17.0            | 0.0        %
```

```
user_003   | 339.3           | 0.0        %
user_004   | 992.0           | 0.0        %
user_005   | 192.0           | 0.0        %
...        | ...             | ...
========================================
Report generated successfully!
```

5.3 Aggregated Results (from report.csv)

User ID	Avg Session (s)	Total Visits	Bounce Rate (%)
user_001	381.5	4	0.0
user_002	17.0	3	0.0
user_003	339.3	3	0.0
user_004	992.0	2	0.0
user_005	192.0	3	0.0
user_006	432.7	3	0.0
user_007	465.7	3	0.0
user_008	222.0	3	0.0
user_009	357.0	3	0.0
user_010	380.3	3	0.0
user_011	150.5	2	0.0
user_012	270.0	2	0.0
user_013	208.5	2	0.0
user_014	441.5	2	0.0
user_015	253.5	2	0.0
user_016	541.0	2	0.0
user_017	137.5	2	0.0
user_018	295.5	2	0.0
user_019	378.5	2	0.0
user_020	355.5	2	0.0

Note: Bounce rate shows 0.0% for all users because the sample.csv provided does not contain any single-value rows. The bounce detection logic (val_count == 1) is correctly implemented in

processor.c and would produce non-zero rates with a dataset that includes single-page-visit rows.

5.4 Process Lifecycle Log (dispatcher output)

```
FIFO created Successfully
Shared Memory Created!
Semaphore Created!
Ingester  PID=4774 exited with status=0
Processor PID=4775 exited with status=0
Reporter  PID=4776 exited with status=0
All Child Processes Finished.
Cleanup Done!
```

5.5 Processor Thread Log (excerpt)

```
Processor Running...  PID=4775  PPID=4773
N=4 threads, Q=10 queue size
Worker Thread 1 Created!  ...  Worker Thread 4 Created!
Reader Thread Running...
Reader got chunk 0 (45 bytes)
Processed: user_001 session=354 bounced=0
Processed: user_003 session=590 bounced=0
...
EOF chunk received!
Poison pill 1 sent!  ...  Poison pill 4 sent!
Worker got poison pill. Exiting...
Results written to shared memory!
Reporter Notified
```

6. Resource Lifecycle and Cleanup

6.1 IPC Resource Lifecycle

Resource	Created By	Destroyed By
/tmp/my_fifo (FIFO)	dispatcher: mkfifo()	dispatcher: unlink() after all waitpids
/my_shm (shared mem)	dispatcher: shm_open() + ftruncate()	dispatcher: munmap() + shm_unlink()
/my_sem (semaphore)	dispatcher: sem_open(O_CREAT)	dispatcher: sem_close() + sem_unlink()
logs/*.log	dispatcher: open() before execvp()	kept on disk for inspection after exit
Thread pool (N threads)	processor: pthread_create() x (N+1)	processor: pthread_join() x (N+1)
sem_empty / sem_full	processor: sem_init()	processor: sem_destroy()
queue_mutex, agg_mutex	processor: static initializers	processor: pthread_mutex_destroy()

6.2 Signal-Driven Graceful Shutdown

Pressing Ctrl-C sends SIGINT to the dispatcher. The handle_sigint handler kills all three children with SIGTERM, then unlinks all IPC resources before calling exit(130). The ingester and processor handle SIGTERM by setting a running flag that breaks their main loops. The ingester writes the EOF chunk before closing the FIFO so the processor does not block waiting for more data. After a clean run, ipcs -m shows no shared-memory segments belonging to the project and ls /tmp/my_fifo confirms the FIFO has been removed.

6.3 Exit Codes

Code	Meaning	Where Used
0	Success	All four components on clean exit
10	Bad command-line arguments	All four components
20	IPC creation failure	dispatcher, processor, reporter
30	Child process died unexpectedly	dispatcher SIGCHLD handler

40	I/O error	ingester, reporter
130	Interrupted by SIGINT	dispatcher handle_sigint
143	Interrupted by SIGTERM	dispatcher handle_sigterm

7. Work Division

Team Member	Contributions
Abdul Rauf (24I-0060)	**processor.c:** Full thread pool. pthread_attr_t setup, reader thread (FIFO reads and queue enqueue), N worker threads (CSV parsing, session/bounce calculation, aggregation updates), both mutexes with all lock/unlock calls, sem_empty/sem_full bounded-buffer pair, poison-pill shutdown, pthread_join loop, and final shared-memory write with sem_post(). **ingester.c:** Directory scanner using opendir/readdir, fgets() file-reading loop, binary ChunkHeader framing, FIFO write calls, EOF sentinel, and signal handlers for SIGTERM and SIGUSR1. **common.h:** All shared data structures (ChunkHeader, UserData, SharedData) and standardised exit-code constants used by every component. **run.sh:** Full Bash orchestrator including getopts parsing, four functions, trap-based cleanup, arithmetic expansion for runtime, and the -c clean option. Passes shellcheck with no warnings.
Abdul Azeem (24I-2013)	**reporter.c:** Shared-memory open and mmap, sem_wait blocking receive, per-user average and bounce-rate calculations, the dup/dup2 stdout-redirect block for report.txt, fopen-based report.csv write, and SIGUSR1 notification to dispatcher via getppid(). **dispatcher.c:** Full master process. Argument parsing, IPC creation (mkfifo, shm_open, sem_open), forking all three children with dup2() log redirection before execvp(), all four signal handlers, waitpid() loop, and complete IPC cleanup on exit. **Testing and debugging:** Ran the pipeline with multiple CSV inputs and thread counts, verified all three log files, confirmed no shared-memory leaks with ipcs -m, and diagnosed and fixed the race condition in the aggregation table.

8. Challenges Faced

8.1 FIFO Blocking on Open

A named FIFO blocks the open() call until both a writer and a reader have opened it. Early in development, launching the ingester before the processor caused the ingester to hang indefinitely because there was no reader on the other end. The fix was to ensure the dispatcher forks and execs the processor before the ingester. Because fork() is nearly instantaneous and the processor opens the FIFO for reading almost immediately, the ingester unblocks as soon as the processor connects.

8.2 Race Conditions in the Aggregation Table

During initial testing with 4 threads, the final user visit counts were sometimes wrong. Multiple threads processing different rows for the same user would simultaneously call find_or_create_user() and increment totals. The fix was to wrap both the find_or_create_user() call and the subsequent table update inside a single agg_mutex lock/unlock block, making the entire lookup-and-write an atomic operation from the perspective of other threads.

8.3 Poison Pill Synchronisation

With N worker threads all blocking on sem_wait(sem_full), a single poison pill only unblocked one thread. The others kept waiting indefinitely and the process never exited. The solution was for the reader thread to enqueue exactly N poison pills after receiving the EOF chunk, each using the same sem_wait(sem_empty) / enqueue / sem_post(sem_full) sequence as real chunks so the queue invariants are never violated during shutdown.

8.4 Restoring stdout After dup2 in Reporter

When we redirected stdout to report.txt with dup2(report_fd, 1), we initially forgot to restore stdout afterwards. Debug messages printed with fprintf(stderr,...) appeared correctly, but subsequent printf() calls after the report section were silently written to the file instead of the terminal. The fix was to save stdout with dup(1) before the redirect and restore it with dup2(saved_stdout, 1) immediately after the report is written.

8.5 Shared Memory Region Too Small

The shared-memory segment was initially ftruncate()d to 4096 bytes. However, sizeof(SharedData) with 100 UserData entries (each containing a 64-byte string and three ints) is approximately 7,600 bytes. On some runs the reporter read garbage data from beyond the mapped region. The fix was to compute the correct size and pass it to both ftruncate() and mmap(). We used 8192 bytes to leave a safe margin.

8.6 CSV Header Row Parsed as a User

The processor treats every CSV line including the header row as a data record. Because atoi("session_length") returns 0, the header produces a user with id "user_id" and session = 0, visits = 1, bounces = 0. This appears in the report. A production fix would detect and skip the first line of each file, but since the specification focuses on OS concepts rather than data validation, the behaviour is documented rather than corrected.

9. Conclusion

The project works end to end. All four processes start, communicate through the IPC channels, and exit cleanly. The dispatcher manages the full lifecycle: creating IPC resources, spawning children with proper log redirection, waiting for them to finish, and destroying everything on exit. The ingester streams CSV data through the FIFO using a binary-header framing protocol. The processor runs it through a bounded-buffer thread pool with correct mutex and semaphore synchronisation and hands the aggregated results to the reporter through shared memory. The reporter blocks on a named semaphore, writes both output files using dup/dup2 as required, and signals the dispatcher when done.

Every OS concept from the course appears in the system. Process creation with fork and exec, IPC through a named FIFO and POSIX shared memory, synchronisation with both named and unnamed semaphores and two mutexes, signals for normal completion and emergency shutdown, file-descriptor redirection with dup and dup2, and a Bash orchestrator with proper error checking, cleanup traps, and a user-friendly summary. After the pipeline exits, ipcs -m is empty and the FIFO is gone.

The Web Clickstream variant is fully implemented. Per-user session length and bounce rate are correctly aggregated across multiple visits by the same user, even when those visits arrive in different CSV rows processed by different worker threads simultaneously. The mutex-protected aggregation table ensures correctness under all concurrency conditions. Building this project made very clear why synchronisation primitives exist: we ran into the aggregation race condition on real hardware and had to reason through it properly to fix it, which was more educational than any textbook description.